

PRACTICAL TASK FOUR

DATABASE & FILESERVER CONNECTIVITY

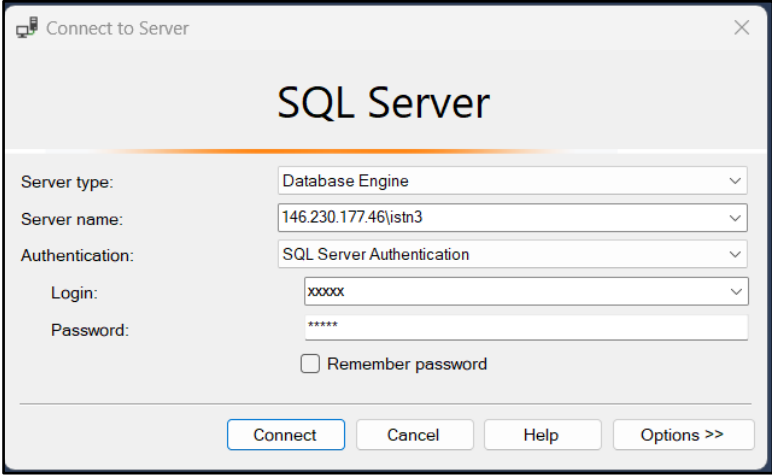
OBJECTIVES

- Establish familiarity with Sql Server development tools
- Test out Sql Server individual accounts on Sql Server
- Create a simple database structures
- Populate database tables with data
- Create a C# application using Visual Studio (VS)
- Establish database connectivity with Sql Server database
- Display data from Sql Server using Windows Forms

PART A

1. Launch Sql Server Management Studio
2. Activate GlobalProtect (University VPN)
3. Establish a connection to the Sql Server
4. Note: The IP address for the SQL Server database is:

146.230.177.46\ISTN3



Connect to Server

SQL Server

Server type: Database Engine

Server name: 146.230.177.46\istn3

Authentication: SQL Server Authentication

Login: xxxxx

Password: *****

☐ Remember password

Connect Cancel Help Options >>

PART B

This is a continuation of Prac 3

In Prac 3 you would have developed an app to display details of customers from a database table named Customer/Cust. Also, Prac 4 required the creation of a database table named Order. In the current prac both of these tables will be used to enable a Customer to record an order (overall order – not the order details). To achieve this functionality, you have to create a new table named **Product**. Add data items to this table. The Product table design and display of sample data is shown below:

Column Name	Data Type	Allow Nulls
ProductID	int	☐
ProductName	varchar(30)	☐
ProductDescription	varchar(50)	☐
ProductPrice	decimal(6, 2)	☐
ProductImage	image	☒

Column Properties	
Identity Specification	Yes
(Is Identity)	Yes
Identity Increment	1
Identity Seed	1

Note: The Image field may be designed to allow for a Null value. This will be updated later.

Add a few records of data into this table: the table shown below is for demonstration purposes – your product table does not have to be so comprehensive!

ProductID	ProductName	ProductDescription	ProductPrice
1	Bread	1 Loaf White Bread	20,00
2	Bread	1 Loaf Brown Bread	18,00
3	Milk	1 Liter Full Cream Milk	15,00
4	Milk	1 Liter Low Fat Milk	14,00
5	Juice	1 Liter Orange Juice	18,00
7	Juice	1 Liter Fruit Juice	20,00
8	Cooldrink	1 Liter Coca Cola	18,00
9	Cooldrink	1 Liter Fanta	16,00
10	Cooldrink	1 Lliter Raspberry	15,00
11	Bread	1/2 Loaf White Bread	10,00
12	Bread	1/2 Loaf Brown Bread	9,00
13	Chocolate	Aero	12,00
14	Chocolate	Flake	12,00
15	Chocolate	Milky Bar	12,00
16	Milk	1/2 Liter Low Fat Milk	10,00
17	Milk	1/2 Liter Full Cream Milk	9,00
18	Chocolate	Kit Kat	12,00
19	Bread	1 Loaf Health Bread	24,00
20	Bread	1/2 Loaf Health Bread	15,00
21	Pie	Mutton Curry	22,00
22	Pie	Chicken and Mushroom	20,00
23	Pie	Vegetable Curry	18,00

Sample data for the **Product** Table

Open up the Prac 3solution and continue with the development of this solution.

Once a customer has been selected, the processing needs to be directed to the Order Tab.

In the Order Tab the customer details will be presented, together with the current date and time. The customer will be provided with an interface to select an item from the Product database table, a quantity for the item and then register a sale transaction. This will generate an Order entry in the Order table

PART C – The Order Tab

In the Order tab add 3 labels – for the current date, customer ID and the customer name

Once a customer is selected, the Order tab should obtain the following details:

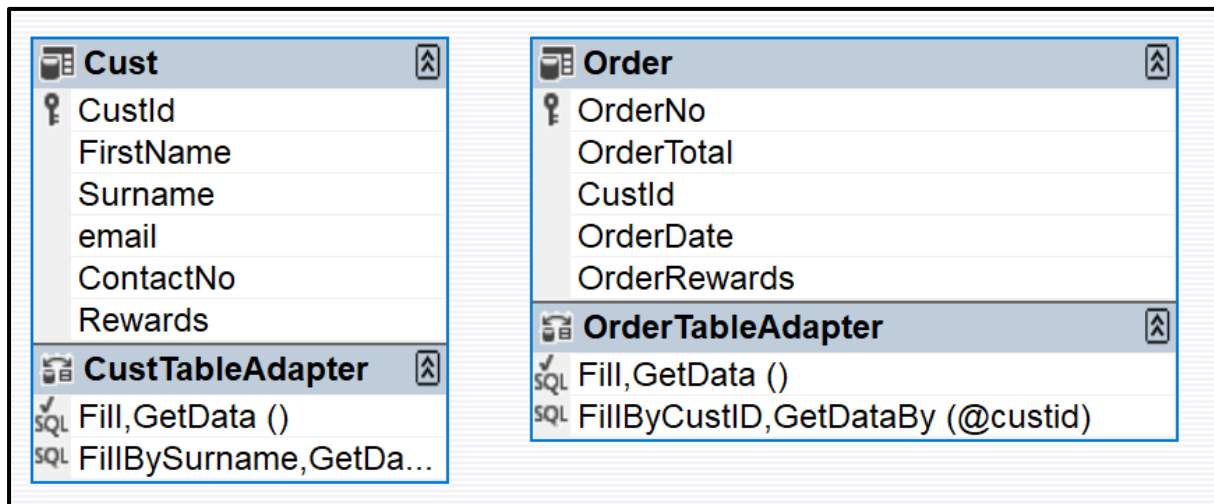
- Current Date - `DateTime.Now.ToShortTimeString();`
- The Selected Customer's ID - `gvCustomer.CurrentRow.Cells[0].Value.ToString();`
- The Selected Customer's Name ...
`gvCustomer.CurrentRow.Cells[1].Value.ToString() + " " + gvCustomer.CurrentRow.Cells[2].Value.ToString();`

When a Customer is selected on the Gridview in the Tab named Customer, the details of that customer should appear as labels in the Tab named order

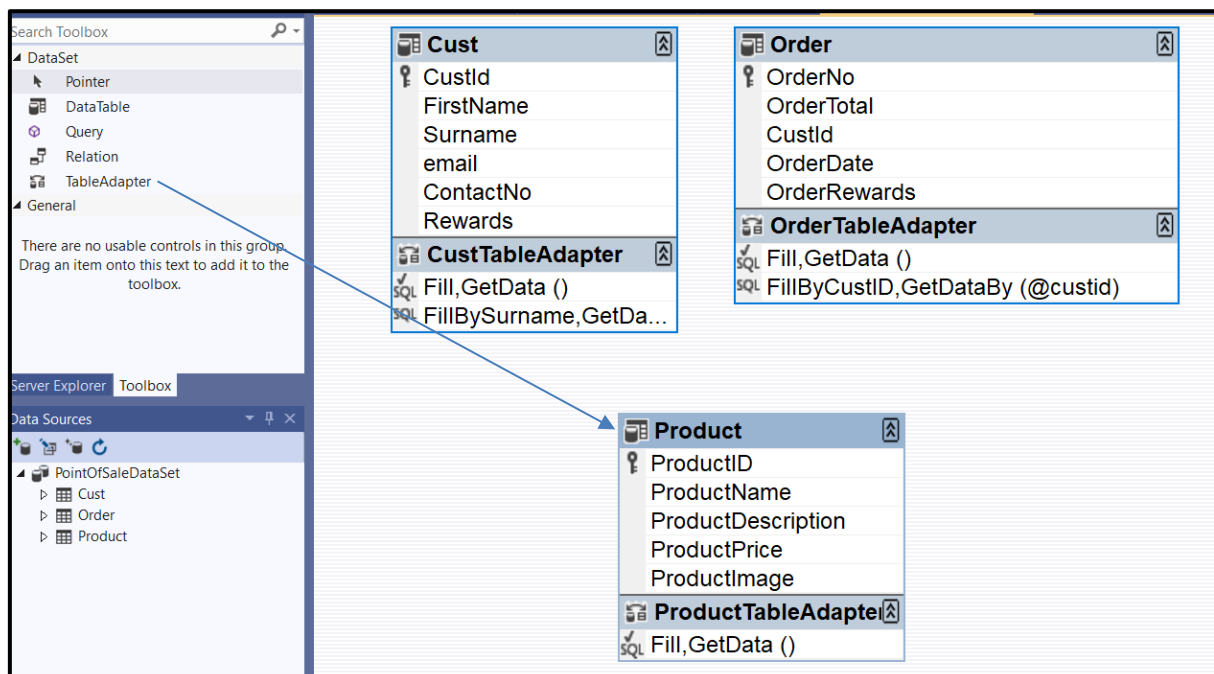
PART D – Processing the Order

We are now in a position to provide an interface whereby the customer can select a product for order. **At this stage we will simply record the entire order and not the inner details of the order (planned for the next prac).**

Currently if you view your Dataset class – you will get the following:

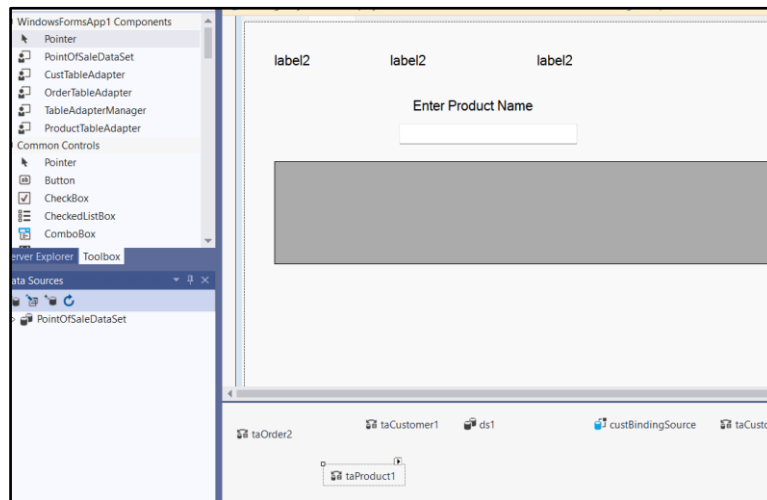


You can add (drag and drop) a TableAdapter class that points to the Product table that you have just created on Sql Server



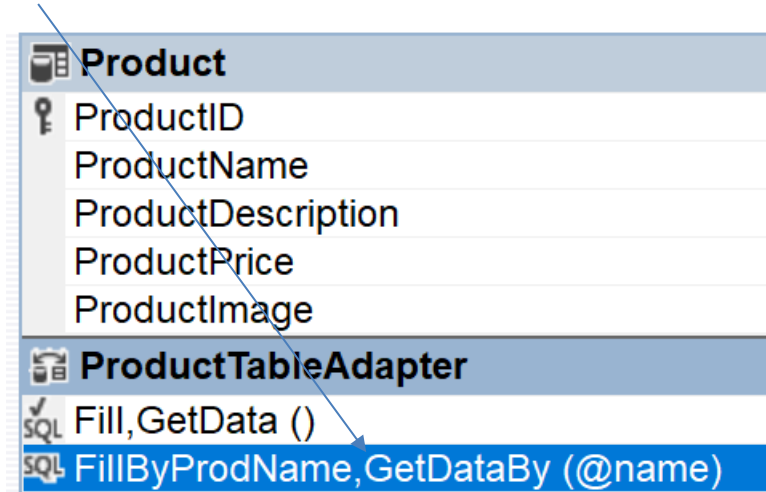
Rebuild the application so that the Table Adapter appears in the Toolbox enabling you to use it

Drag the Product Table Adapter onto the tray of the form so that it becomes an object; also add a textbox and Gridview to display the products

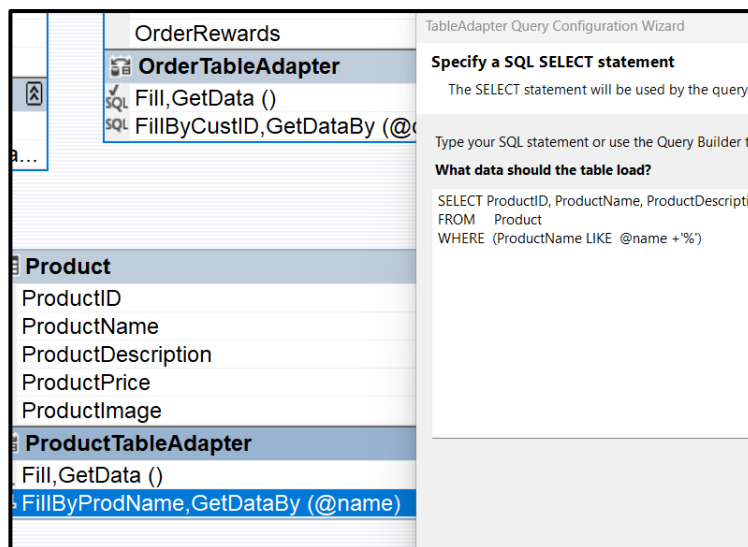


Use the same logic that was used to display customer details. However, this time around you will be displaying the names of products that match the product name that is typed into the textbox

First, add a method onto the table adapter to enable a Like search on the product



```
SELECT ProductID, ProductName, ProductDescription, ProductPrice, ProductImage
FROM Product
WHERE (ProductName LIKE @name +'%')
```



On the Forms app, write in the following code so that when characters are entered into the textbox, the gridview becomes populated with products that match the characters entered in the textbox. The display in the Order tab will now look like...

Enter Product Name

	ProductID	ProductName	ProductDescription	ProductPrice
▶	8	Cooldrink	1 Liter Coca Cola	18,00
	9	Cooldrink	1 Liter Fanta	16,00
	10	Cooldrink	1 Liter Raspberry	15,00
	13	Chocolate	Aero	12,00

The next challenge is to add an interface option to select the quantity

One suggestion is to add an unbounded Combobox as a new column in the gridview. You can now fill in quantity values so that the user can select the quantity of items being purchased.

Form1
Customer Order

label2 label2 label2

Enter Product Name
c

	ProductID	ProductName	ProductDescription	ProductPrice	Quantity
✓	8	Cooldrink	1 Liter Coca Cola	18,00	4
	9	Cooldrink	1 Liter Fanta	16,00	1
	10	Cooldrink	1 Liter Raspberry	15,00	2
	13	Chocolate	Aero	12,00	3

4
5
6
7
8
9
10

Add a textbox to enable a capture of the total value of the items that have been selected for purchase (those where a quantity value is greater than zero) and a button to enable recording of the product and a button to finalise the Order amount.

So, the interface looks like....

label2 label2 label2

Enter Product Name

	ProductID	ProductName	ProductDescription	ProductPrice	Quantity
*					✓

Order Final Total 0

Add to Order

When the application is run....

The screenshot shows a web application interface with a header bar containing 'Customer' and 'Order' tabs. Below the header, the current time is 11:42, the Customer ID is 2, and the customer name is Miley Cyrus. There is a text input field labeled 'Enter Product Name' with the letter 'c' entered. Below this is a table of products:

	ProductID	ProductName	ProductDescription	ProductPrice	Quantity
	8	Cooldrink	1 Liter Coca Cola	18,00	2
▶	9	Cooldrink	1 Liter Fanta	16,00	3
	10	Cooldrink	1 Liter Raspberry	15,00	
	13	Chocolate	Aero	12,00	

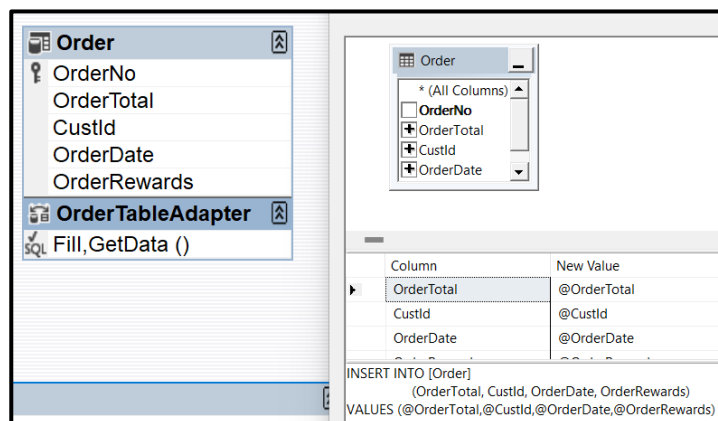
At the bottom, there is a label 'Order Final Total' followed by a text box containing '84,00' and a blue button labeled 'Add to Order'.

PART E – Capturing the Order

This is quite simple – it is a case of Inserting the new Order as a new record into the Order table. This entails recording the date, Customer ID and the Order total. The recording of the individual items in the order will be done in a later prac.

Insert the new record into the Order table:

If you look at the Order Table Adapter class inside the dataset, you will be able to find an Insert method that is already pre-written



All that is required is to activate this Insert Method!

- Issue a dialog message to the user to determine confirmation of sale
- If the response is confirm, then use the Insert method of the Order table adapter and insert the following:
 - Total amount of the order – from the textbox
 - The Customer ID – from the label
 - The current date and time – *DateTime.Now*

- To verify that the order has been recorded, re-fill the gridview from the tab named Customer to show the new order that has just been recorded

```
private void btnFinalOrder_Click(object sender, EventArgs e)
{
    DialogResult dialogResult = MessageBox.Show("Confirm Order to "+lblCustName.Text+"
    for an amount of " + txtTotalOrderAmt.Text, "Order Confirmation",
    MessageBoxButtons.YesNo);
    if (dialogResult == DialogResult.Yes)
    {
        taOrder1.Insert(Convert.ToDecimal(txtTotalOrderAmt.Text), Convert.ToInt32(lblCustID.Text), DateTime.Now,null);
        taOrder1.FillByCustID(ds1.Order, (int)gvCustomer.CurrentRow.Cells[0].Value);
    }
    else if (dialogResult == DialogResult.No)
    {
        gvProd.Rows.Clear();
        txtTotalOrderAmt.Text = "";
    }
}
```

Sample Run:

Customer Order

2
2025/04/14 Miley Cyrus

Enter Name of Product

c

ProductID	ProductName	ProductDescr	ProductPrice	Qty
8	Cooldrink	1 Liter Coca...	18,00	
9	Cooldrink	1 Liter Fanta	16,00	2
10	Cooldrink	1 Liter Ras...	15,00	
13	Chocolate	Aero	12,00	3
14	Chocolate	Flake	12,00	
15	Chocolate	Milky Bar	12,00	
18	Chocolate	Kit Kat	12,00	
24	Chips	Simba Chips	12,00	

Confirm Order

Order Total R104,00

Add to Order

Once the Order has been confirmed, you can revert to the Customer Order to confirm that the new transaction has been recorded in the Order Table ...and when you go back to the Customer tab...you will see a record of the recent order

Customer Order

Enter the Customer Surname

c

	CustId	FirstName	Surname	email	ContactNo
▶	2	Miley	Cyrus	cyrus@gm...	23456
	3	Black	Coffee	coffee@ya...	32456
	7	Pryankra	Chopra	chopra@g...	32467
	9	Jimmy	Carr	carr@yaho...	22113
	32	Charlie	Chaplin	chaplinchar...	44232

2

R1 993,00

	OrderNo	OrderTotal	CustId	OrderDate	Order
	197	84,00	2	2023/05/04 13:31	
	200	60,00	2	2023/05/04 17:07	
	228	200,00	2	2024/05/11 20:57	
	241	104,00	2	2025/04/14 11:20	
*					

*****END OF PRAC 4*****